A Cloud-Native Logistics and Supply Chain Management Platform Using Service-Oriented Microservices Architecture

B. Hemalatha
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad–500043, Telangana, India

M. MahaLakshmi
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad–500043, Telangana, India

B. Bhavya Sai Sri
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad–500043, Telangana, India

***Abstract*—The logistics and supply chain industry requires efficient coordination among product management, inventory operations, order processing, shipment tracking, and delivery management. Traditional monolithic systems can introduce scalability, maintenance, and integration challenges as operational requirements increase. This paper presents LogiSphere, a cloud-native logistics and supply chain management platform developed using a Service-Oriented Architecture (SOA) and microservices-based design.**

The proposed platform separates business functionalities into independent services developed using Java and Spring Boot. Spring Cloud Gateway provides centralized API routing, while Netflix Eureka supports service discovery. Apache Kafka is incorporated for asynchronous communication between selected services, and MySQL provides relational data persistence. A React-based frontend enables users to interact with the platform through a unified interface.

The architecture emphasizes loose coupling, modularity, independent service development, and cloud deployment. The paper describes the system architecture, service interactions, implementation methodology, security approach, and deployment considerations. The proposed system provides a practical implementation of SOA and cloud-native design principles in logistics management.

***Index Terms*—Service-Oriented Architecture, Cloud Computing, Microservices, Logistics Management, Spring Boot, API Gateway, Service Discovery, Apache Kafka, Docker, Supply Chain Management.**

I. Introduction

Logistics and supply chain management involve the coordination of multiple operations, including product management, inventory control, order processing, shipment handling, and delivery management. Efficient communication among these operations is essential for maintaining operational accuracy and reducing processing delays.

Traditional monolithic applications combine multiple business functionalities into a single application. Although this approach may be suitable for small systems, it can make independent development, maintenance, and scaling more difficult as the application grows.

Service-Oriented Architecture provides an approach in which business functionalities are exposed through well-defined services. Microservices extend this concept by organizing an application into independently deployable services with clearly defined responsibilities.

Cloud computing provides infrastructure and deployment capabilities that support distributed applications. Combining cloud-native practices with SOA principles provides a foundation for developing modular and scalable enterprise applications.

This paper presents LogiSphere, a logistics and supply chain management platform that integrates SOA principles with cloud-native technologies. The system separates business operations into independent Spring Boot services and uses centralized API routing, service discovery, database separation, and asynchronous messaging.

II. Problem Statement

Conventional logistics management applications may face several operational challenges:

- Disconnected product, inventory, order, and shipment management operations.
- Difficulty maintaining communication between different business modules.
- Limited flexibility when modifying individual application functionalities.
- Challenges in scaling application components independently.
- Security and authentication management across multiple application modules.
- Difficulties coordinating asynchronous business operations.

LogiSphere addresses these challenges by introducing a modular service-oriented architecture in which individual services manage specific business capabilities.

III. Objectives

The major objectives of the proposed system are:

1) To develop a centralized logistics management platform.
2) To implement independent backend services using Spring Boot.
3) To provide centralized API routing through an API Gateway.
4) To implement dynamic service discovery using Netflix Eureka.
5) To provide authentication using JSON Web Tokens (JWT).
6) To support product, inventory, order, shipment, and delivery management.
7) To integrate asynchronous messaging using Apache Kafka.
8) To support containerization and cloud deployment.

IV. Related Work

Cloud computing has changed the way software applications are developed, deployed, and managed. Armbrust et al. [1] discussed the opportunities and challenges associated with cloud computing, including scalability, availability, and resource provisioning.

Service-Oriented Architecture emphasizes the organization of software functionality into reusable and interoperable services. Erl [2] describes the principles and design considerations associated with service-oriented systems.

Newman [3] discusses microservices as an architectural approach in which applications are divided into small, independently deployable services.

These concepts provide the architectural foundation for LogiSphere. The proposed platform applies service decomposition, centralized request routing, service discovery, and asynchronous communication to logistics management operations.

V. Proposed System

LogiSphere is designed as a cloud-native logistics management platform using a microservices-based Service-Oriented Architecture.

The system contains a React frontend and eight backend services. Each backend service is responsible for a specific business functionality.

The API Gateway acts as the common entry point for frontend requests. Eureka provides service discovery, while Kafka supports asynchronous communication between selected services.

MySQL is used for relational data storage, with separate logical databases allocated to the business services.

A. System Architecture

The proposed architecture consists of the following layers:

- **Presentation Layer:** React-based frontend for user interaction.
- **Gateway Layer:** Spring Cloud Gateway for request routing.
- **Service Discovery Layer:** Netflix Eureka for service registration and discovery.
- **Business Layer:** Independent Spring Boot microservices.
- **Messaging Layer:** Apache Kafka and ZooKeeper for asynchronous event communication and coordination.
- **Data Layer:** MySQL databases for persistent business information.

Fig. 1 illustrates the logical organization of the platform. The actual runtime communication and deployment configuration may differ from this logical architecture.

B. Microservices Description

The platform consists of eight backend services.

1) Authentication Service: The Authentication Service manages user registration, login, and authentication. It uses Spring Security and JWT-based authentication mechanisms.

2) API Gateway: The API Gateway provides a centralized entry point for client requests. It routes requests to the appropriate backend service based on configured URL paths.

3) Eureka Server: The Eureka Server provides service registration and discovery capabilities. It maintains a registry of available backend services.

4) Product Service: The Product Service manages product-related information and exposes APIs for product operations.

5) Inventory Service: The Inventory Service manages inventory information and supports stock-related operations.

6) Order Service: The Order Service manages order creation and order-related processing. It communicates with other business services as required.

7) Shipment Service: The Shipment Service manages shipment-related information and supports shipment processing.

8) Delivery Service: The Delivery Service handles delivery-related operations and delivery assignment functionality.

C. Service Configuration

Table I presents the backend service configuration identified in the project.

VI. Service Communication

LogiSphere uses synchronous and asynchronous communication mechanisms.

A. Synchronous Communication

HTTP-based REST APIs are used for direct communication between frontend and backend components and between services where required.

The API Gateway routes requests according to configured URL paths.

Eureka supports service discovery by maintaining the registry of registered services.

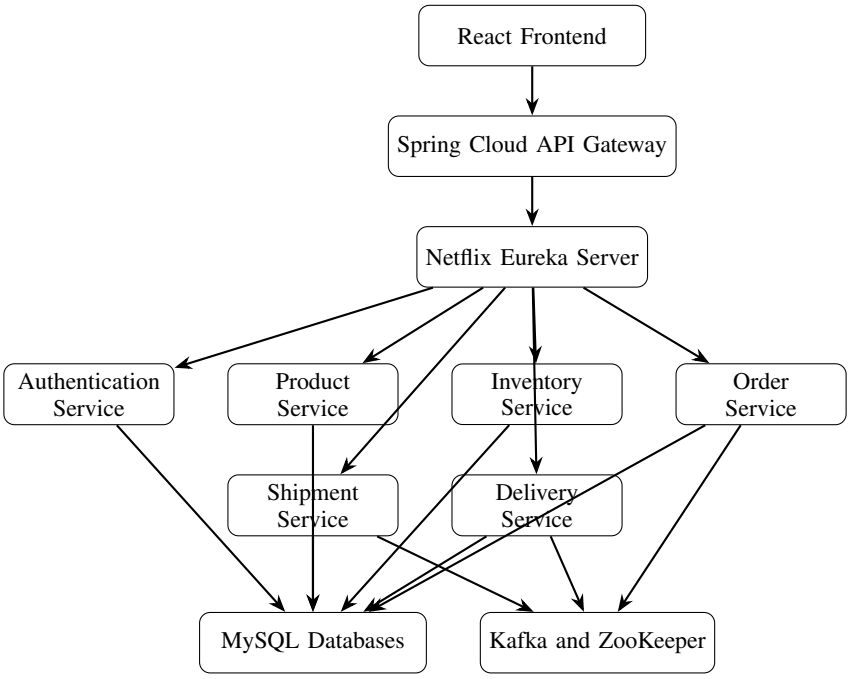


Fig. 1. Logical architecture of the LogiSphere platform.

TABLE I
LOGISPHERE BACKEND SERVICES

Service	Responsibility	Port
API Gateway	Request routing	8080
Authentication	User authentication	8082
Product	Product management	8083
Inventory	Stock management	8084
Order	Order processing	8085
Shipment	Shipment management	8086
Delivery	Delivery management	8087
Eureka	Service discovery	8761

B. Asynchronous Communication

Apache Kafka is incorporated to support event-based communication between selected services.

This approach allows event producers and consumers to exchange messages without requiring every operation to be performed through a direct synchronous request.

ZooKeeper is included in the configured Kafka infrastructure for coordination.

VII. DATABASE DESIGN

MySQL is used as the relational database management system.

The project initialization script defines six logical databases:

- logisphere_auth_db
- logisphere_product_db
- logisphere_inventory_db
- logisphere_order_db
- logisphere_shipment_db
- logisphere_delivery_db

The logical separation of databases supports service-level data organization.

The entity relationships, table structures, constraints, and indexing strategies should be documented using the actual database schema.

VIII. SECURITY ARCHITECTURE

Security is an important component of the LogiSphere platform.

The Authentication Service implements registration and login operations.

After successful authentication, a JSON Web Token is generated. The token can be used by the client when accessing protected APIs.

The application also uses Spring Security configuration and Cross-Origin Resource Sharing (CORS) settings to manage frontend-backend communication.

The production deployment should use secure environment variables for database credentials and JWT signing secrets.

IX. CLOUD DEPLOYMENT AND CONTAINERIZATION

Docker is used to package backend services into container images.

The project contains individual Dockerfiles for the eight backend services.

The frontend is deployed using Vercel, while Render is the selected target platform for backend deployment.

The deployment configuration is maintained through a Render Blueprint file.

Cloud deployment requires appropriate environment variables, database connectivity, network configuration, health checks, and service discovery settings.

Free-tier infrastructure may introduce resource limitations and cold-start delays. Therefore, deployment behavior must be evaluated using the actual hosting configuration.

X. Implementation Methodology

The implementation follows a modular development approach.

First, the frontend interface is developed using React. Backend business capabilities are then separated into independent Spring Boot services.

The API Gateway is configured to route requests to backend services. Eureka provides service discovery in the distributed deployment configuration.

MySQL databases are initialized for the individual business modules. Kafka is configured for selected asynchronous communication workflows.

Dockerfiles are provided to package the backend services for container-based deployment.

The system is maintained using Git and GitHub for source code management.

XI. Evaluation Methodology

The proposed platform should be evaluated using functional and deployment-level tests.

The following metrics can be considered:

A. Response Time

Response time measures the time required for the system to process a request.

$$T_r = T_{\text{response}} - T_{\text{request}} \tag{1}$$

B. Service Availability

Availability can be calculated as:

$$A = \frac{T_{\text{operational}}}{T_{\text{total}}} \times 100 \tag{2}$$

C. Throughput

Throughput measures the number of successfully processed requests per unit time.

$$Throughput = \frac{N_{\text{successful}}}{T_{\text{measurement}}} \tag{3}$$

D. Functional Testing

The following operations should be tested:

- User registration.
- User login and JWT generation.
- Product API operations.
- Inventory API operations.
- Order processing.
- Shipment management.
- Delivery assignment.
- Inter-service communication.

Actual response times, throughput values, availability measurements, and test outcomes must be obtained from experiments before reporting quantitative results.

XII. Results and Discussion

The LogiSphere implementation demonstrates the decomposition of logistics operations into independent backend services.

The logical architecture separates user authentication, product management, inventory, order processing, shipment management, and delivery operations.

The API Gateway provides a centralized request-routing mechanism, while Eureka supports service discovery in the configured microservices environment.

Kafka provides an asynchronous messaging mechanism for selected service interactions.

The React frontend provides a unified interface for interacting with the backend system.

The cloud deployment configuration supports container-based deployment. However, complete cloud availability and performance must be reported only after successful deployment and testing of the required services.

XIII. Advantages

The proposed platform provides the following advantages:

- Modular service-oriented architecture.
- Separation of business responsibilities.
- Independent development of backend services.
- Centralized API routing.
- Service discovery support.
- Asynchronous messaging support.
- Logical database separation.
- Containerization support.
- Improved maintainability.

XIV. Limitations and Future Scope

The platform introduces distributed-system complexity, including network communication, service dependencies, monitoring requirements, and deployment configuration.

Future enhancements may include GPS-based shipment tracking, intelligent route optimization, predictive inventory management, demand forecasting, automated notifications, and advanced logistics analytics.

Additional work may also investigate fault tolerance, distributed tracing, load balancing, and performance optimization.

XV. Conclusion

This paper presented LogiSphere, a cloud-native logistics and supply chain management platform developed using Service-Oriented Architecture and microservices principles.

The platform separates business functionalities into eight Spring Boot services and integrates them using an API Gateway, service discovery, relational databases, and asynchronous messaging.

The architecture demonstrates the application of loose coupling, modularity, service autonomy, and cloud deployment practices within a logistics management environment.

LogiSphere provides a foundation for further development of scalable and maintainable logistics applications. Future

work will focus on comprehensive cloud deployment evaluation, performance measurement, intelligent logistics features, and improved fault tolerance.

REFERENCES

[1] M. Armbrust et al., "A view of cloud computing," Communications of the ACM, vol. 53, no. 4, pp. 50–58, 2010, doi: 10.1145/1721654.1721672.

[2] T. Erl, *Service-Oriented Architecture: Concepts, Technology, and Design*. Upper Saddle River, NJ, USA: Prentice Hall, 2005.

[3] S. Newman, *Building Microservices*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[4] R. T. Fielding, *Architectural Styles and the Design of Network-Based Software Architectures*. Ph.D. dissertation, University of California, Irvine, 2000.